

TASK 01 REPORT

INTERNSHIP
CYBERSECURITY

20
26

SUBMITTED BY: Muhammad Saim Ali Khan
ROLE: Cyber Security Analyst Intern
ID: DHC-4044

WEEK 1: SECURITY ASSESSMENT REPORT

Intern: Muhammad Saim Ali Khan

Role: Cyber Security Analyst

DHC: 4044

Date: May 8, 2026

1. APPLICATION SETUP

During the initial phase, I established a local testing environment to simulate a real-world web application audit.

- **Source:** Cloned the OWASP Juice Shop repository from GitHub.
- **Environment:** Installed dependencies via `npm install` and launched the server using `npm start`.
- **Accessibility:** Verified the application was fully functional at `http://localhost:3000`, focusing on high-risk areas including the **Login**, **Signup**, and **Search** functions.

2. VULNERABILITY ASSESSMENT METHODOLOGY

I employed a hybrid testing strategy combining automated scanning and manual penetration testing to ensure maximum coverage.

- **Automated Scanning:** Utilized **OWASP ZAP 2.16.1** to perform an active scan of the local environment.
- **Manual Testing:** Used **Browser Developer Tools (F12)** and manual injection payloads to confirm vulnerabilities that automated tools might overlook.

3. KEY FINDINGS & VULNERABILITIES IDENTIFIED

Vulnerability	Risk Level	Method of Discovery	Description
SQL Injection (SQLi)	Critical	Manual Payload	Successfully bypassed the Admin login using the ' OR 1=1 -- payload.
Cross-Site Scripting (XSS)	High	Manual Payload	Confirmed script execution in the Search bar using an <code><iframe src="javascript:alert(1)"></code> payload.
Broken Access Control	High	Manual Traversal	Accessed confidential documents and hidden administrative pages (Score Board).
Missing Security Headers	Medium	OWASP ZAP	Missing CSP , X-Frame-Options , and X-Content-Type headers, increasing the risk of clickjacking and XSS.
Cookie Misconfiguration	Low	OWASP ZAP	Cookies identified without the SameSite attribute or HttpOnly flag.
Information Disclosure	Informational	OWASP ZAP	Server leaks version info (Express) and contains suspicious developer comments.

4. DETAILED ANALYSIS (TECHNICAL PROOF)

A. Authentication Bypass (SQL Injection)

By entering a tautology (`' OR 1=1 --`) into the email field, I successfully manipulated the backend SQL query to return a `TRUE` value, granting me full administrative access without a valid password.

B. Automated Scan Alerts (ZAP)

The ZAP scan identified that the application is a **Modern Web Application** but lacks essential defensive headers. Specifically, the lack of a **Content Security Policy (CSP)** allows the browser to execute untrusted scripts.

5. PROPOSED AREAS OF IMPROVEMENT (WEEK 2 PLAN)

Following the assessment, I have identified the following remediation steps to be implemented next week:

1. **Sanitize Inputs:** Use the `validator` library to prevent XSS payloads from executing.
 2. **Password Hardening:** Replace plain-text storage with salted hashes using `bcrypt`.
 3. **Header Security:** Implement `helmet.js` to automatically set **CSP**, **X-Frame-Options**, and **MIME-sniffing** protections.
 4. **Cookie Security:** Configure cookies with `SameSite=Strict` and `HttpOnly` flags to prevent CSRF and session hijacking.
-

6. CONCLUSION

The Week 1 assessment confirms that the application is currently susceptible to critical injection attacks. While automated tools found several configuration issues, manual testing revealed the most significant business risks (Admin bypass). Remediation will focus on securing the "Data" and "Application" tiers in the coming week.